

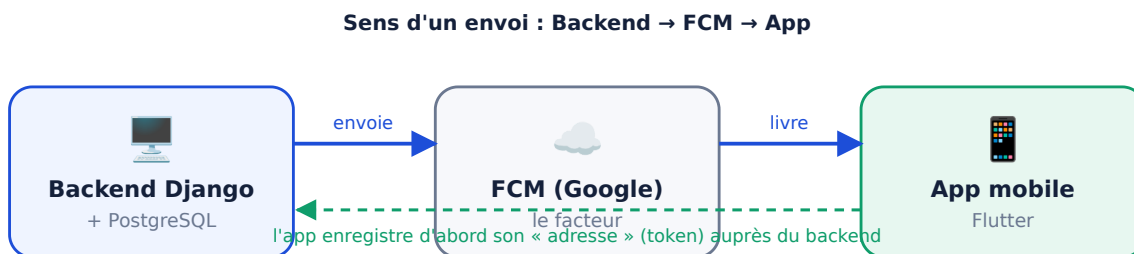
Notifications Push — le mécanisme complet

Comment un événement du serveur arrive, en temps réel, sous forme de notification sur le téléphone du membre — et **pourquoi** chaque pièce existe.

1. L'idée en une phrase

Ton serveur ne peut pas « appeler » directement un téléphone (IP qui change, réseau mobile, appareil endormi). On passe donc par un **intermédiaire de confiance** — **Firebase Cloud Messaging (FCM)**, chez **Google** — qui garde une connexion permanente avec chaque appareil Android et se charge de livrer le message pour toi.

Il y a donc **trois acteurs** et le message voyage toujours dans le même sens à l'émission :



Le serveur remet le message au « facteur » FCM, qui connaît l'adresse (le token) du téléphone et le livre.

Pourquoi un intermédiaire ? Un téléphone n'a pas d'adresse fixe joignable et dort la plupart du temps. FCM maintient *une seule* connexion permanente par appareil (économie de batterie), met les messages en file quand l'appareil est hors-ligne, et les livre dès son réveil. Ton serveur, lui, ne parle jamais directement au téléphone.

2. Les deux clés (et pourquoi il en faut deux)

Le point qui trouble le plus : il y a **deux fichiers de configuration différents**, pour deux rôles opposés.

Fichier	Où	Rôle	Analogie
<code>google-services.json</code>	Dans l' app (<code>android/app/</code>)	Identité du client : dit à l'app « tu appartiens à CE projet Firebase » (sender ID, clés publiques). Généré automatiquement par <code>flutterfire configure</code> .	La carte SIM du téléphone : elle l'identifie sur le réseau.
<code>service-account</code> (JSON)	Sur le serveur (<code>FCM_CREDENTIALS_JSON</code>)	Autorité du serveur : prouve que TON backend a le droit d'envoyer des messages <i>au nom du projet</i> . Tu le copies depuis la console Firebase.	Le tampon officiel qui autorise le facteur à accepter tes colis.

Pourquoi séparer ? L'app est distribuée à des milliers de téléphones : on n'y met jamais un secret capable d'envoyer des notifications (n'importe qui décompilerait l'APK et spammerait). L'app ne porte donc qu'une **identité** (publique, sans pouvoir). Le pouvoir d'émettre reste sur le serveur, protégé, via le **service account**.

3. Le scénario complet, de bout en bout

On suit un message concret : « quelqu'un a répondu au commentaire du membre ». Trois phases : **A. Enregistrement** (une fois), **B. Émission** (à chaque événement), **C. Réception**.

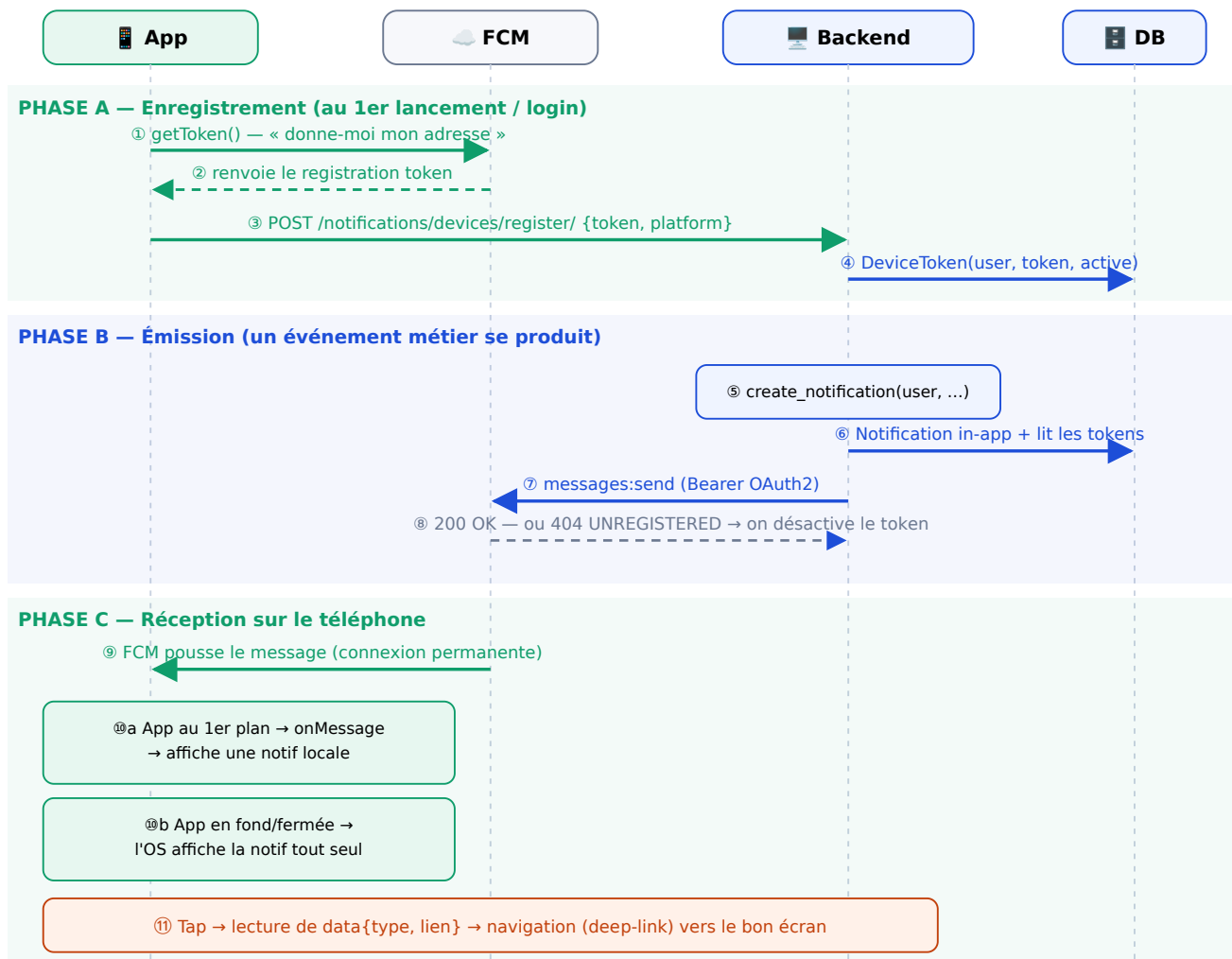


Diagramme de séquence du flux complet. Les flèches pleines = requêtes ; pointillées = réponses.

4. Chaque étape expliquée (le « pourquoi »)

Phase A — Enregistrement

- 1 **getToken()** — au démarrage, l'app initialise Firebase et demande son **registration token** : une chaîne unique qui identifie *cette installation sur ce téléphone*.
Pourquoi : c'est « l'adresse postale » du device pour ton app. Sans elle, impossible de cibler l'envoi.
- 2 **FCM renvoie le token** — Google le génère et le garde en mémoire côté FCM.
- 3 **L'app l'envoie à TON backend** (`syncToken()` après le login) → `POST /notifications/devices/register/`.
Pourquoi : FCM connaît le token mais pas *quel membre* c'est. Seul ton backend fait le lien `user → token`.
- 4 **Le backend stocke un** `DeviceToken(user, token, platform, active=true)`.
Un même membre peut avoir plusieurs appareils → plusieurs tokens.

Phase B — Émission

- 1 **Un événement métier** (réponse à un commentaire, campagne, paiement validé...) appelle `create_notification(user, ...)` — **un point d'entrée unique**.
- 2 Le backend enregistre la **notification in-app** (celle de la cloche 🛎), puis lit les `DeviceToken` actifs du membre.
Pourquoi le push est « en plus » : la notif in-app est la source de vérité ; le push n'est qu'un **canal secondaire** d'alerte.
- 3 **send_push_to_user → FCM** : pour chaque token, un `POST` vers `fcm.googleapis.com/v1/projects/<id>/messages:send`, avec un jeton **OAuth2 court** dérivé du service account.
Pourquoi OAuth2 : l'ancienne « clé serveur » est dépréciée. FCM v1 exige un jeton signé, à durée limitée, issu du service account → plus sûr.
- 4 **FCM répond** : `200` = pris en charge. `404 / UNREGISTERED` = token mort (app désinstallée, données effacées) → le backend **désactive** ce `DeviceToken`.
Pourquoi nettoyer : éviter d'envoyer indéfiniment vers des appareils disparus.

Phase C — Réception

- 1 **FCM livre** le message au téléphone via sa connexion permanente (même endormi, il le réveille).
- 2 **Deux cas d'affichage** :
 - **App au premier plan** → Android n'affiche *rien* automatiquement ; c'est `onMessage` qui déclenche une **notification locale** (via `flutter_local_notifications`).
 - **App en fond / fermée** → l'OS affiche la notif tout seul à partir du bloc `notification{title,body}`.
Pourquoi cette différence : quand l'app est ouverte, l'utilisateur regarde déjà l'écran — c'est à l'app de décider comment (ou si) alerter.
- 3 **Au tap** : `onMessageOpenedApp` / `getInitialMessage` lit le bloc `data{type, lien}` et **navigue** vers le bon écran (deep-link).
Pourquoi deux blocs (`notification` + `data`) : `notification` sert à l'affichage par l'OS ; `data` sert au **routing** interne. On envoie les deux.

5. Cycle de vie du token (points souvent oubliés)

- **Rotation** : le token peut changer (réinstallation, effacement des données, refresh de sécurité). → écouter `onTokenRefresh` et **ré-enregistrer** automatiquement.
- **Logout** : appeler `clearToken()` → `unregister` → on désactive le token, sinon l'ancien membre recevrait les notifs du suivant sur le même téléphone.
- **Permission (Android 13+)** : il faut demander `POST_NOTIFICATIONS` à l'exécution, sinon aucune notif ne s'affiche même si tout le reste marche.
- **Best-effort partout** : un push cassé (FCM indisponible, token invalide) ne doit **jamais** bloquer un flux métier — d'où le « no-op loggé » quand ce n'est pas configuré.

6. Où en est le projet aujourd'hui

Les « tuyaux » sont posés et **dormants** : tant qu'aucune clé FCM n'est là, tout est un no-op sûr.

Élément	État	Détail
Backend — modèle <code>DeviceToken</code>	POSÉ	user, token, platform, active, last_seen.
Backend — endpoints register/unregister	POSÉ	<code>/notifications/devices/register unregister/</code> .
Backend — <code>send_push_to_user</code>	POSÉ	Déjà appelé dans <code>create_notification</code> (best-effort).
Backend — <code>_deliver()</code> (envoi réel FCM v1)	STUB	Renvoie <code>False</code> ; à implémenter (OAuth2 + messages:send).
Backend — credentials <code>FCM_CREDENTIALS_JSON</code>	À POSER	Service account, en env prod.
Mobile — <code>PushRegistrationService</code>	POSÉ	<code>syncToken</code> / <code>clearToken</code> prêts...
Mobile — appel au login/logout	À BRANCHER	Le service existe mais n'est appelé nulle part.
Mobile — <code>PushTokenProvider</code>	NO-OP	<code>NoopPushTokenProvider</code> renvoie <code>null</code> .
Mobile — deps Firebase + <code>google-services.json</code>	ABSENT	Généré par <code>flutterfire configure</code> .

7. Les étapes pour l'activer (checklist)

① Prérequis Firebase (une fois)

- Projet Firebase + app Android (package `com.gathefinance.gathe_finance`).
- `flutterfire configure` → génère `lib/firebase_options.dart` + `android/app/google-services.json`.
- Console Firebase → Comptes de service → **Générer une clé privée** (JSON) → pour le backend.

② Mobile (Flutter)

- Deps : `firebase_core`, `firebase_messaging`, `flutter_local_notifications`.
- Gradle : plugin `com.google.gms.google-services`.
- `Firebase.initializeApp()` dans `main()`.
- **FcmPushTokenProvider** (`getToken` + `onTokenRefresh`) → remplace le Noop.
- Permission `POST_NOTIFICATIONS` ; canal de notification Android + icône.
- Brancher `syncToken()` (login) / `clearToken()` (logout).
- Handlers : foreground (notif locale), background (fonction top-level), tap → deep-link.

③ Backend

- Implémenter `_deliver()` : OAuth2 (service account) → `POST .../messages:send`.
- Désactiver les tokens invalides (404 / UNREGISTERED).
- Poser `FCM_CREDENTIALS_JSON` (+ project id) en env prod.

④ Test bout-en-bout

- Créer une notif → réception sur un appareil **connecté à internet** → le tap ouvre le bon écran.

Rappel : l'appareil de test est habituellement **hors-ligne** — le push ne pourra se vérifier que sur un téléphone **connecté**.